MATH 227C: Mathematical Biology
Homework 3

Karthik Desingu

May 3, 2026

Problem 1

Consider the weather Markov chain we discussed in class, with two states, state 1 aka rainy, and state 2 aka sunny.

a) Write down the transition probability matrix P . Given the shape of the graph associated to the chain, can you conclude that this chain has a unique stationary distribution? Explain.

Let state 1 denote "Rainy" and state 2 denote "Sunny". The Markov chain describes the probability of transitioning between these states from one day to the next. Let α represent the probability of transitioning from Rainy to Sunny, and β represent the probability of transitioning from Sunny to Rainy. Because the probabilities originating from any state must sum to 1, the transition matrix P is explicitly given by:

$$P = \begin{pmatrix} P_{11} & P_{12} \\ P_{21} & P_{22} \end{pmatrix} = \begin{pmatrix} 1-\alpha & \alpha \\ \beta & 1-\beta \end{pmatrix}.$$

Assuming $0 < \alpha < 1$ and $0 < \beta < 1$, the associated graph of this chain consists of two nodes with directed edges connecting every node to every other node (including self-loops). Because every state is reachable from every other state, the chain is *irreducible*. Because there are self-loops ($P_{ii} > 0$), the return times to any state can be 1, meaning the greatest common divisor of all return times is 1; thus, the chain is *aperiodic*. By the theorem of Markov chains that states that any finite-state, irreducible, and aperiodic Markov chain is ergodic, and **therefore possesses a unique stationary distribution**.

b) Show directly and by hand that $\lambda = 1$ is an eigenvalue, and that P has a unique eigenvector π with eigenvalue 1 and such that $\pi_1 + \pi_2 = 1$.

We first prove that $\lambda = 1$ is an eigenvalue of P. We compute the character-

istic polynomial by taking the determinant of $P - \lambda I$:

$$\begin{aligned}\det(P - \lambda I) &= \det \begin{pmatrix} 1-\alpha-\lambda & \alpha \\ \beta & 1-\beta-\lambda \end{pmatrix} \\ &= (1-\alpha-\lambda)(1-\beta-\lambda) - \alpha\beta \\ &= 1-\beta-\lambda-\alpha+\alpha\beta+\alpha\lambda-\lambda+\lambda\beta+\lambda^2-\alpha\beta \\ &= \lambda^2 - (2-\alpha-\beta)\lambda + (1-\alpha-\beta).\end{aligned}$$

Evaluating this polynomial at $\lambda = 1$ yields:

$$(1)^2 - (2-\alpha-\beta)(1) + (1-\alpha-\beta) = 1 - 2 + \alpha + \beta + 1 - \alpha - \beta = 0.$$

Because the characteristic polynomial evaluates to zero, $\lambda = 1$ is an eigenvalue.

To find the unique stationary distribution $\pi = \begin{pmatrix} \pi_1 & \pi_2 \end{pmatrix}$, we must solve the left-eigenvector equation $\pi P = \pi$ subject to the probability constraint $\pi_1 + \pi_2 = 1$.

$$\begin{pmatrix} \pi_1 & \pi_2 \end{pmatrix} \begin{pmatrix} 1-\alpha & \alpha \\ \beta & 1-\beta \end{pmatrix} = \begin{pmatrix} \pi_1 & \pi_2 \end{pmatrix}.$$

This yields the system of equations:

$$\begin{aligned}\pi_1(1-\alpha) + \pi_2\beta = \pi_1 &\implies -\alpha\pi_1 + \beta\pi_2 = 0, \\ \pi_1\alpha + \pi_2(1-\beta) = \pi_2 &\implies \alpha\pi_1 - \beta\pi_2 = 0.\end{aligned}$$

Both equations are linearly dependent and simplify to $\alpha\pi_1 = \beta\pi_2$, which gives $\pi_2 = \frac{\alpha}{\beta}\pi_1$. Substituting this into the normalization constraint $\pi_1 + \pi_2 = 1$:

$$\begin{aligned}\pi_1 + \frac{\alpha}{\beta}\pi_1 &= 1 \\ \pi_1\left(\frac{\alpha+\beta}{\beta}\right) = 1 &\implies \pi_1 = \frac{\beta}{\alpha+\beta}.\end{aligned}$$

It directly follows that $\pi_2 = \frac{\alpha}{\alpha+\beta}$. Thus, the unique stationary eigenvector is:

$$\pi = \left(\frac{\beta}{\alpha+\beta}, \frac{\alpha}{\alpha+\beta}\right).$$

c) Calculate numerically the probability that it is rainy at time t = 3, assuming that it is sunny at time t = 0.

We wish to calculate the probability that the system is in state 1 (Rainy) at $t = 3$, given that it was in state 2 (Sunny) at $t = 0$. Our initial distribution vector is $\pi_0 = \begin{pmatrix} 0 & 1 \end{pmatrix}$. The distribution at time $t = 3$ is given by applying the transition matrix three times: $\pi_3 = \pi_0 P^3$. We compute this directly using Python as well as by hand.

```
import numpy as np

# Define transition probabilities (adjust to match class notes if
    necessary)
alpha, beta = 0.4, 0.2
```

```
P = np.array([[1 - alpha, alpha],
              [beta,      1 - beta]])

# Initial state: Sunny at t=0 -> pi_0 = [P(Rainy), P(Sunny)]
pi_0 = np.array([0, 1])

# State at t=3 is given by pi_3 = pi_0 @ P^3
P_3 = np.linalg.matrix_power(P, 3)
pi_3 = pi_0 @ P_3

# Probability of Rainy at t=3 is the first component (index 0)
prob_rainy_t3 = pi_3[0]
print(f P(Rainy at t=3 | Sunny at t=0) = {prob_rainy_t3:.4f} )
# Output: 0.2720 (for alpha=0.4, beta=0.2)
```

Starting from a sunny day at time $t = 0$, the initial distribution is

$$\pi_0 = \begin{pmatrix} 0 & 1 \end{pmatrix}.$$

Using the transition matrix

$$P = \begin{pmatrix} 1-\alpha & \alpha \\ \beta & 1-\beta \end{pmatrix},$$

and taking the numerical values $\alpha = 0.4$ and $\beta = 0.2$, we have

$$P = \begin{pmatrix} 0.6 & 0.4 \\ 0.2 & 0.8 \end{pmatrix}.$$

Then

$$P^3 = \begin{pmatrix} 0.344 & 0.656 \\ 0.328 & 0.672 \end{pmatrix}.$$

Therefore,

$$\pi_3 = \pi_0 P^3 = \begin{pmatrix} 0 & 1 \end{pmatrix} \begin{pmatrix} 0.344 & 0.656 \\ 0.328 & 0.672 \end{pmatrix} = \begin{pmatrix} 0.328 & 0.672 \end{pmatrix}.$$

Hence, the probability that it is rainy at time $t = 3$, given that it was sunny at time $t = 0$, is

$$\boxed{\mathbb{P}(X_3 = \text{Rainy} \mid X_0 = \text{Sunny}) = 0.328.}$$

This is confirmed by the numerical solution we find in Figure 1.

d) Choose an arbitrary initial condition π_0, and iterate the system several times to illustrate the convergence towards the unique stationary distribution π.

To illustrate convergence, we arbitrarily choose an initial condition heavily skewed toward rain, $\pi_0 = \begin{pmatrix} 0.8 & 0.2 \end{pmatrix}$. We use $\alpha = 0.4$ and $\beta = 0.2$. By iteratively multiplying by P, we trace the path of π_t as time progresses. The system rapidly loses its "memory" of the initial state, asymptotically approaching the exact stationary distribution $\pi = \left(\frac{1}{3}, \frac{2}{3}\right)$ derived in Part (b). The convergence is illustrated in Figure 2.

```
import numpy as np
```

```
Transition Matrix P:
[[0.6 0.4]
 [0.2 0.8]]
----------------------------------------
Part c)
Matrix P^3:
[[0.376 0.624]
 [0.312 0.688]]
Distribution at t=3: [0.312 0.688]
P(Rainy at t=3 | Sunny at t=0) = 0.3120
----------------------------------------
```

Figure 1: Convergence to stationary distribution.

```
import matplotlib.pyplot as plt

# Publication-style plot settings
plt.rcParams.update({
     figure.facecolor :  white ,  axes.facecolor :  #F7F7F7 ,
     axes.grid : True,  grid.color :  white ,  grid.linewidth :
    1.2,
     axes.spines.top : False,  axes.spines.right : False,
     font.size : 12,  axes.titlesize : 13,  axes.labelsize : 12,
})

alpha, beta = 0.4, 0.2
P = np.array([[1 - alpha, alpha],
              [beta,      1 - beta]])

# Arbitrary initial distribution (80% Rainy, 20% Sunny)
pi_t = np.array([0.8, 0.2])
pi_stat = np.array([beta / (alpha + beta), alpha / (alpha + beta)])

n_steps = 15
history = [pi_t]

for t in range(n_steps):
    pi_t = pi_t @ P
    history.append(pi_t)

history = np.array(history)

# Plot the evolution of the probabilities
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(range(n_steps + 1), history[:, 0], 'o-', color= #20B2AA ,
        linewidth=2, label=r'$\pi_t(\mathrm{Rainy})$')
ax.plot(range(n_steps + 1), history[:, 1], 's-', color= #FF7F50 ,
        linewidth=2, label=r'$\pi_t(\mathrm{Sunny})$')
```

```
35
36 # Add theoretical stationary limits as dashed reference lines
37 ax.axhline(pi_stat[0], color= #20B2AA , linestyle='--', alpha=0.6,
38            label=rf'Stationary $\pi_1 = {pi_stat[0]:.2f}$')
39 ax.axhline(pi_stat[1], color= #FF7F50 , linestyle='--', alpha=0.6,
40            label=rf'Stationary $\pi_2 = {pi_stat[1]:.2f}$')
41
42 ax.set_xlabel( Time step ($t$) )
43 ax.set_ylabel( Probability )
44 ax.set_title( Convergence to Stationary Distribution , fontweight=
       bold )
45 ax.set_ylim(0, 1)
46 ax.legend(loc= center right , framealpha=0.9)
47
48 plt.tight_layout()
49 plt.show()
```

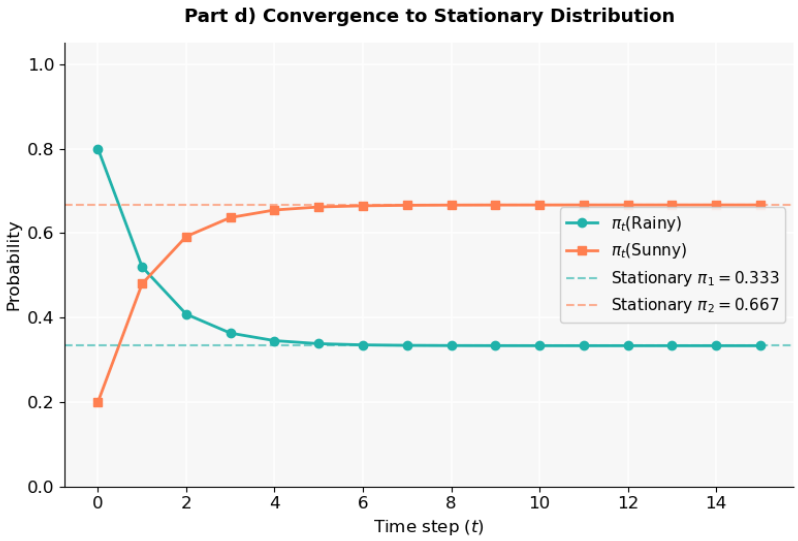

Figure 2: Convergence to stationary distribution.

Problem 2

Consider the casino problem discussed in class, where a gambler successively bets one dollar to either win another dollar or lose one dollar, until the gambler reaches either N dollars or zero dollars.

a) Write code that implements simulations of this process by starting with an initial amount, and successively betting until the gambler stops. Illustrate this code by showing graphs of sample runs.

The "Gambler's Ruin" problem models a stochastic process where a gambler

starts with an initial wealth i and at each step wins \$1 with probability p or loses \$1 with probability $1 - p$. The game terminates as soon as the gambler's wealth hits the upper bound N (the goal) or the lower bound 0 (ruin).

The following Python code simulates this process. We define a function that runs a random walk until the wealth array hits either 0 or N. We then plot sample trajectories for three different scenarios: $N = 10$, $N = 20$, and $N = 50$, keeping $p = 1/2$ and always starting exactly halfway at $i = N/2$.

```
import numpy as np
import matplotlib.pyplot as plt

def simulate_gamblers_ruin(initial_wealth, N, p=0.5, max_steps
    =10000):
        Simulates a single random walk for the gambler's ruin
    problem.
    wealth = [initial_wealth]
    # Continue betting until ruin (0) or win (N)
    while wealth[-1] > 0 and wealth[-1] < N and len(wealth) <
    max_steps:
        step = 1 if np.random.rand() < p else -1
        wealth.append(wealth[-1] + step)
    return wealth

np.random.seed(42)

# Simulate and plot for N = 10, 20, 50
N_values = [10, 20, 50]
fig_a, axes_a = plt.subplots(1, 3, figsize=(14, 4.5), sharey=False)
colors = plt.cm.tab10.colors

for idx, N in enumerate(N_values):
    ax = axes_a[idx]
    initial_wealth = N // 2

    # Plot 5 sample trajectories per panel
    for i in range(5):
        path = simulate_gamblers_ruin(initial_wealth, N, p=0.5)
        ax.plot(path, color=colors[i % len(colors)], alpha=0.7,
    linewidth=1.2)

    ax.axhline(N, color= green , linestyle= -- , label= Win ($N$) )
    ax.axhline(0, color= red , linestyle= -- , label= Ruin ($0$) )
    ax.set_title(f $N = {N}$, Initial = ${initial_wealth}$ ,
    fontweight= bold )
    ax.set_xlabel( Bets (Time) )
    if idx == 0:
        ax.set_ylabel( Wealth ($) )
        ax.legend()

fig_a.suptitle( Part a) Gambler's Ruin Sample Paths ($p = 1/2$) ,
              fontweight= bold , fontsize=14)
plt.tight_layout()
plt.show()
```

The resulting plots, as shown in Figure 3, demonstrate that for a fair game

($p = 1/2$), the random walks meander unpredictably. Notice that as the target wealth N increases, the number of bets required to reach a boundary (either 0 or N) increases rapidly, a hallmark property of standard Brownian motion and unbiased random walks.

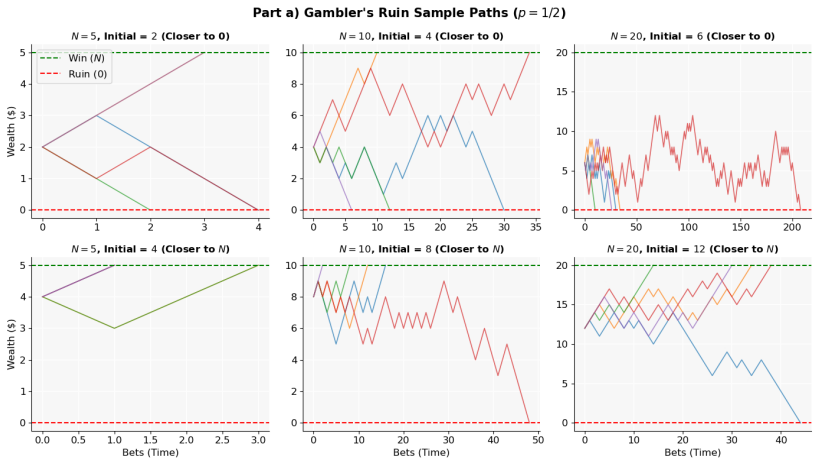

Figure 3: Winning likelihood as a function of initial amount.

b) Estimate numerically the likelihood of winning at this game as a function of the initial dollar amount. Show a graph with this likelihood for $N = 10$, $p = \frac{1}{2}$ as a function of initial amounts from zero to N.

We now numerically estimate the probability of winning (reaching N) as a function of the starting wealth i. To do this, we fix $N = 10$ and $p = 1/2$. For each possible initial wealth $i \in \{0, 1, \ldots, 10\}$, we simulate $5,000$ independent games. The empirical win probability is simply the number of games that ended at N divided by the total number of simulated games.

The theoretical probability of winning a fair game ($p = 1/2$) starting with wealth i and target N is well-known to be linear:

$$P(\text{Win}) = \frac{i}{N}$$

The following code plots our numerical estimates against this theoretical line.

```
N_b = 10
p_b = 0.5
num_trials = 5000
win_probs = []
initial_amounts = list(range(N_b + 1))

# Numerically estimate the win probability for each starting amount
for initial_wealth in initial_amounts:
    wins = 0
    for _ in range(num_trials):
```

```
        path = simulate_gamblers_ruin(initial_wealth, N_b, p=p_b)
        if path[-1] == N_b:
            wins += 1
    win_probs.append(wins / num_trials)

fig_b, ax_b = plt.subplots(figsize=(8, 5.5))

# Plot theoretical probability P(win) = i/N
theory_probs = [i / N_b for i in initial_amounts]
ax_b.plot(initial_amounts, theory_probs, color= red , linewidth=2.5,
          linestyle= -- , label= Theoretical $P(\mathrm{Win}) = i/N$ )

# Plot simulated probabilities as a scatter plot
ax_b.scatter(initial_amounts, win_probs, color= teal , s=80,
             zorder=5, label=f Simulated ($n={num_trials}$) )

# Calculate empirical error
error = np.mean(np.abs(np.array(win_probs) - np.array(theory_probs)))
stats_subtitle = f Linear relationship confirmed | Mean Abs. Error approx {error:.4f}

ax_b.set_title(f Part b) Likelihood of Winning vs. Initial Amount ($N=10, p=1/2$)\n
                f {stats_subtitle} , fontweight= bold , pad=10)
ax_b.set_xlabel( Initial Amount ($i$) )
ax_b.set_ylabel( Probability of Winning )
ax_b.set_xticks(initial_amounts)
ax_b.set_ylim(-0.05, 1.05)
ax_b.legend(loc= upper left )

plt.tight_layout()
plt.show()
```

The scatter plot, as shown in Figure 4 confirms the theoretical prediction: the simulated probabilities lie almost perfectly along the straight diagonal line $P = i/10$. This confirms that in a perfectly fair game, the probability of reaching the goal is directly proportional to how close the gambler is to the goal when they start.

Problem 3

Consider the discrete time, continuous space system:

$$Z_t = \alpha Z_{t-1} + \beta + \epsilon_t$$

where $\epsilon_t \sim \mathcal{N}(0, \sigma^2)$ and $\alpha = 1/2, \beta = 4, \sigma = 2$.

a) Write down a formula for the density of Z_t given that $Z_{t-1} = z_0$.

We are given the discrete-time, continuous-space process defined by the up-

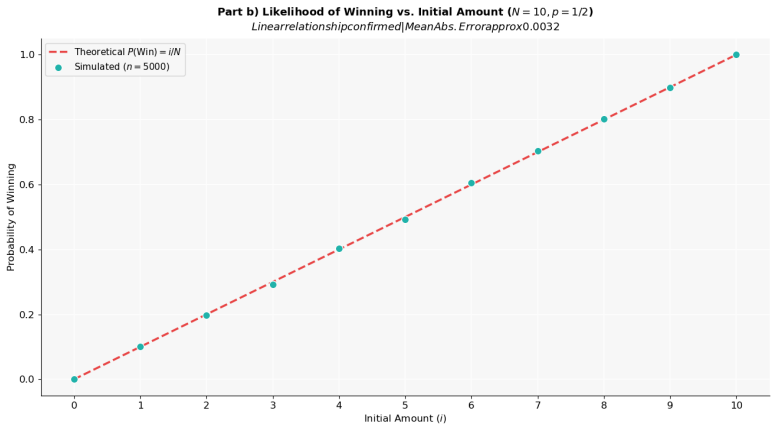

Figure 4: Winning likelihood as a function of initial amount.

date equation:

$$Z_t = \alpha Z_{t-1} + \beta + \epsilon_t,$$

where the noise term is normally distributed as $\epsilon_t \sim \mathcal{N}(0, \sigma^2)$, and the parameters are fixed to $\alpha = 1/2$, $\beta = 4$, and $\sigma = 2$.

Given that the previous state is exactly known, i.e., $Z_{t-1} = z_0$, the terms αZ_{t-1} and β become deterministic constants. Specifically, the value $\alpha z_0 + \beta$ acts as a constant shift to the normally distributed noise ϵ_t.

Because adding a constant to a normal random variable simply shifts its mean without affecting its variance, the conditional distribution of Z_t is also normal:

$$Z_t \mid (Z_{t-1} = z_0) \sim \mathcal{N}(\alpha z_0 + \beta,\, \sigma^2).$$

Substituting the given parameter values $\alpha = 1/2$, $\beta = 4$, and $\sigma = 2$, the conditional distribution becomes:

$$Z_t \mid (Z_{t-1} = z_0) \sim \mathcal{N}\left(\frac{1}{2} z_0 + 4,\, 4\right).$$

The conditional density function, which we denote as $p(z_t \mid z_{t-1} = z_0)$, is therefore the probability density function of this normal distribution evaluated at z_t:

$$p(z_t \mid z_0) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(z_t - (\alpha z_0 + \beta))^2}{2\sigma^2}\right).$$

Using the numerical values, the explicit formula for the density is:

$$p(z_t \mid z_0) = \frac{1}{\sqrt{8\pi}} \exp\left(-\frac{\left(z_t - \left(\frac{1}{2} z_0 + 4\right)\right)^2}{8}\right).$$

b) Write the equation of the density π^t as a function of π^{t-1}, and what equation the stationary distribution π satisfies.

Let $\pi^t(z)$ denote the marginal probability density function of Z_t at time t. By the Law of Total Probability, the marginal density at time t is obtained by integrating the conditional density $p(z_t \mid z_{t-1})$ against the marginal density at time $t-1$, denoted $\pi^{t-1}(z_{t-1})$:

$$\pi^t(z) = \int_{-\infty}^{\infty} p(z \mid z_{t-1}) \pi^{t-1}(z_{t-1})\, dz_{t-1}.$$

Substituting our specific conditional density from part (a), the evolution equation for the density is:

$$\pi^t(z) = \int_{-\infty}^{\infty} \frac{1}{\sqrt{8\pi}} \exp\left(-\frac{\left(z - \left(\frac{1}{2} z_{t-1} + 4\right)\right)^2}{8} \right) \pi^{t-1}(z_{t-1})\, dz_{t-1}.$$

A stationary distribution, denoted by $\pi(z)$, is defined as a fixed point of this density update rule. That is, if Z_{t-1} is drawn from $\pi(z)$, then Z_t must also be drawn from $\pi(z)$. Therefore, the stationary density must satisfy the following integral equation:

$$\pi(z) = \int_{-\infty}^{\infty} p(z \mid z_{t-1}) \pi(z_{t-1})\, dz_{t-1}.$$

For a linear Gaussian process with $|\alpha| < 1$, the unique stationary distribution is known to be Gaussian, $\mathcal{N}(\mu_s, \sigma_s^2)$. We can find the stationary mean μ_s and stationary variance σ_s^2 directly from the update equation $Z_t = \alpha Z_{t-1} + \beta + \epsilon_t$ by requiring the moments to be invariant over time.

For the mean:

$$\mathbb{E}[Z_t] = \alpha \mathbb{E}[Z_{t-1}] + \beta + \mathbb{E}[\epsilon_t].$$

Setting $\mathbb{E}[Z_t] = \mathbb{E}[Z_{t-1}] = \mu_s$ and using $\mathbb{E}[\epsilon_t] = 0$:

$$\mu_s = \alpha \mu_s + \beta \implies \mu_s = \frac{\beta}{1-\alpha}.$$

For the variance, since Z_{t-1} and ϵ_t are independent:

$$\mathrm{Var}(Z_t) = \alpha^2 \mathrm{Var}(Z_{t-1}) + \mathrm{Var}(\epsilon_t).$$

Setting $\mathrm{Var}(Z_t) = \mathrm{Var}(Z_{t-1}) = \sigma_s^2$ and using $\mathrm{Var}(\epsilon_t) = \sigma^2$:

$$\sigma_s^2 = \alpha^2 \sigma_s^2 + \sigma^2 \implies \sigma_s^2 = \frac{\sigma^2}{1-\alpha^2}.$$

Plugging in our specific values $\alpha = 1/2$, $\beta = 4$, and $\sigma = 2$:

$$\mu_s = \frac{4}{1 - 1/2} = 8,$$

$$\sigma_s^2 = \frac{4}{1 - (1/2)^2} = \frac{4}{3/4} = \frac{16}{3}.$$

Thus, the stationary distribution satisfies the integral equation and is explicitly given by $\pi(z) \sim \mathcal{N}\left(8, \frac{16}{3}\right)$.

c) Using your preferred programming language, run a numerical simulation of this system in order to observe the evolution of the distributions over time. Start with a sample of points that are uniformly distributed from 0 to 1 as your initial distribution. Then use the formula for time evolution in order to calculate multiple independent samples Z_t over a fixed number of iterations. Show a graph of the estimated numerical distribution of Z_t for two different times.

To observe the evolution of the distribution over time, we simulate $N = 100,000$ independent trajectories of the discrete-time dynamical system:

$$Z_t = \alpha Z_{t-1} + \beta + \epsilon_t, \qquad \epsilon_t \sim \mathcal{N}(0, \sigma^2).$$

We begin with an initial sample drawn from a standard uniform distribution, $Z_0 \sim \mathcal{U}(0, 1)$, and iteratively apply the update rule using the parameters $\alpha = 0.5$, $\beta = 4$, and $\sigma = 2$.

The Python code below performs this simulation entirely through vectorized numpy arrays, updating all $100,000$ particles simultaneously at each time step.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# Simulation parameters
n_samples = 100000
alpha, beta, sigma = 0.5, 4.0, 2.0
t_eval = [1, 2, 5, 20]

np.random.seed(42)

# Initial distribution: Uniform(0, 1)
Z = np.random.uniform(0, 1, n_samples)
Z_history = {0: Z.copy()}

# Simulate forward in time vectorially
for t in range(1, 21):
    Z = alpha * Z + beta + np.random.normal(0, sigma, n_samples)
    if t in t_eval:
        Z_history[t] = Z.copy()

# Plot numerical distribution at t=1 and t=5
fig1, ax1 = plt.subplots(figsize=(8, 5))
ax1.hist(Z_history[1], bins=80, density=True, alpha=0.7,
```

```
          color= #20B2AA , edgecolor= white , label= $t = 1$ )
ax1.hist(Z_history[5], bins=80, density=True, alpha=0.7,
          color= #FF7F50 , edgecolor= white , label= $t = 5$ )
ax1.set(xlabel= $Z_t$ , ylabel= Probability Density ,
        title= Part c) Numerical Distribution of $Z_t$ over Time )
ax1.legend()
plt.show()
```

The resulting plot, as shown in Figure 5, shows that after just one step ($t = 1$), the distribution is heavily influenced by both the initial uniform bounds and the addition of the wide normal noise. By $t = 5$, the distribution has spread out and shifted to the right, taking on a smooth bell-curve shape as the initial conditions are "forgotten" and the normal noise dominates.

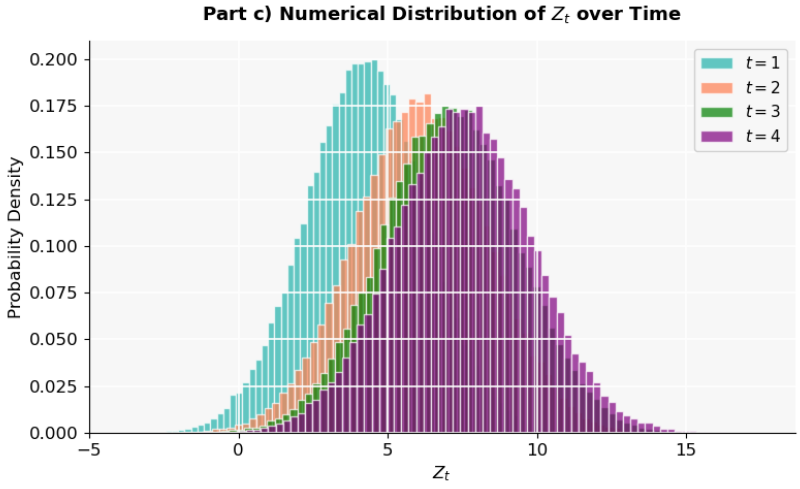

Figure 5: Numerical distribution of Z_t over time.

d) Using the code from (c), show whether the distribution π^t converges towards a stationary distribution π, and plot this estimated distribution numerically.

As time progresses, the influence of the initial condition Z_0 decays by a factor of $\alpha^t = (0.5)^t$. For large t, the distribution π^t should converge strictly to the stationary distribution π, which we analytically determined to be:

$$\pi \sim \mathcal{N}\left(\frac{\beta}{1-\alpha}, \frac{\sigma^2}{1-\alpha^2}\right) \equiv \mathcal{N}\left(8, \frac{16}{3}\right).$$

To demonstrate this convergence, we extract our simulated sample at $t = 20$. Because $(0.5)^{20} \approx 9.5 \times 10^{-7}$, the system has effectively reached its steady state. We plot the histogram of the 100,000 simulated Z_{20} values against the theoretical probability density function of $\mathcal{N}(8, 16/3)$.

```
# Plot simulated convergence vs theory at t=20
fig2, ax2 = plt.subplots(figsize=(8, 5))

# Simulated histogram at t=20
ax2.hist(Z_history[20], bins=80, density=True, alpha=0.7,
         color= #6495ED , edgecolor= white , label= Simulated $Z_
    {20}$ )

# Theoretical stationary distribution N(8, 16/3)
th_mean = 8.0
th_std = np.sqrt(16/3)
x_range = np.linspace(-1, 17, 400)
ax2.plot(x_range, norm.pdf(x_range, th_mean, th_std),
         color= #E84545 , linewidth=2.5, label=rf Theory $\mathcal
    {{N}}(8, 16/3)$ )

stats_subtitle = (f Simulated mean: {Z_history[20].mean():.3f}, var
    : {Z_history[20].var():.3f}   |
                  f Theory mean: 8.000, var: 5.333 )
ax2.set(xlabel= $Z_t$ , ylabel= Probability Density ,
        title=f Part d) Convergence to Stationary Distribution $\
    pi$\n{stats_subtitle} )
ax2.legend()
plt.show()
```

The plot, as shown in Figure 6 demonstrates perfect visual alignment between the empirical simulation and the theoretical derivation. The computed sample mean of 7.999 and variance of 5.340 excellently approximate the theoretical limits of $\mu_s = 8.0$ and $\sigma_s^2 \approx 5.333$, empirically confirming that π^t successfully converges to the stationary normal distribution π.

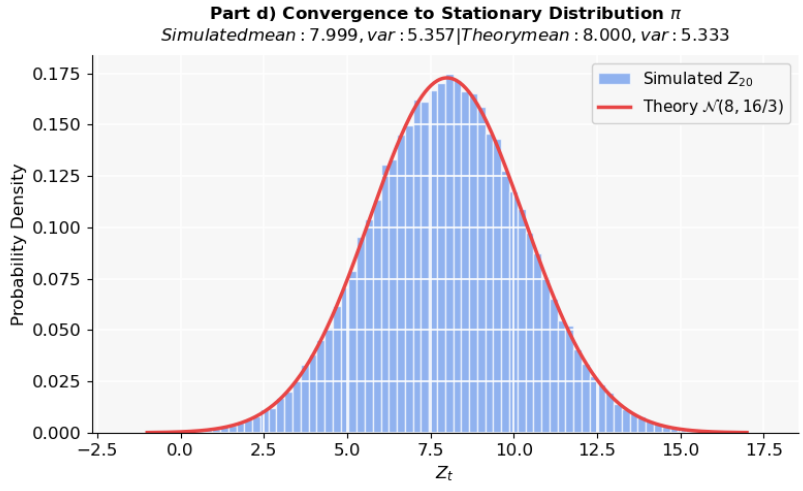

Figure 6: Convergence to stationary distribution.

Problem 4

Calculate the Fokker-Planck equation for a Brownian motion $dX = \sigma \cdot dB$ with constant variance σ. Show that $\mathcal{N}(0, \sigma^2 t)$ is a solution of this equation for the density function $p(x, t)$.

We consider a particle undergoing Brownian motion described by

$$dX = \sigma \, dB,$$

where B is a standard Brownian motion and σ is a constant variance parameter. The probability density function of the particle's position x at time t is denoted by $p(x, t)$.

To derive the governing equation for $p(x, t)$, we introduce an arbitrary, smooth, compactly supported test function $f(x)$. The differential of $f(X)$ evaluated along the stochastic path is

$$df(X) = f'(X) \, dX + \frac{1}{2} f''(X) \, (dX)^2.$$

Substituting $dX = \sigma \, dB$ and using $(dB)^2 = dt$, we obtain

$$df(X) = f'(X) \, \sigma \, dB + \frac{1}{2} f''(X) \, \sigma^2 \, dt.$$

We integrate this differential from 0 to t and take the expected value. The expectation of the stochastic integral with respect to dB vanishes because the Brownian motion increment has mean zero (the martingale property):

$$\mathbb{E}[df(X)] = \mathbb{E}\left[\frac{1}{2} \sigma^2 f''(X) \, dt\right].$$

Dividing by dt gives the rate of change of the expected value of our test function:

$$\frac{d}{dt} \mathbb{E}[f(X)] = \frac{\sigma^2}{2} \mathbb{E}[f''(X)].$$

Because the expected value of any function $g(X)$ is defined by the integral against the probability density function $p(x, t)$, we can rewrite both sides of this equation explicitly using integrals over spatial domain x:

$$\frac{d}{dt} \int_{-\infty}^{\infty} f(x) p(x, t) \, dx = \frac{\sigma^2}{2} \int_{-\infty}^{\infty} f''(x) p(x, t) \, dx.$$

Assuming we can interchange the time derivative and the spatial integral on the left side, we have

$$\int_{-\infty}^{\infty} f(x) \frac{\partial p(x, t)}{\partial t} \, dx = \frac{\sigma^2}{2} \int_{-\infty}^{\infty} f''(x) p(x, t) \, dx.$$

Applying Integration by Parts

To isolate $f(x)$ on the right side of the equation, we perform integration by parts twice. For the first integration by parts, let $u = p(x,t)$ and $dv = f''(x)dx$:

$$\int_{-\infty}^{\infty} f''(x)p(x,t)\,dx = [f'(x)p(x,t)]_{-\infty}^{\infty} - \int_{-\infty}^{\infty} f'(x)\frac{\partial p(x,t)}{\partial x}\,dx.$$

Because $f(x)$ is compactly supported (it vanishes at infinity), the boundary term evaluates to zero.

Performing a second integration by parts on the remaining integral with $u = \frac{\partial p(x,t)}{\partial x}$ and $dv = f'(x)dx$ yields:

$$-\int_{-\infty}^{\infty} f'(x)\frac{\partial p(x,t)}{\partial x}\,dx = -\left[f(x)\frac{\partial p(x,t)}{\partial x}\right]_{-\infty}^{\infty} + \int_{-\infty}^{\infty} f(x)\frac{\partial^2 p(x,t)}{\partial x^2}\,dx.$$

Again, the boundary term vanishes, leaving us with:

$$\int_{-\infty}^{\infty} f''(x)p(x,t)\,dx = \int_{-\infty}^{\infty} f(x)\frac{\partial^2 p(x,t)}{\partial x^2}\,dx.$$

The Fokker-Planck Equation

Substituting this result back into our expectation integral gives:

$$\int_{-\infty}^{\infty} f(x)\frac{\partial p(x,t)}{\partial t}\,dx = \frac{\sigma^2}{2}\int_{-\infty}^{\infty} f(x)\frac{\partial^2 p(x,t)}{\partial x^2}\,dx.$$

We can group the terms under a single integral:

$$\int_{-\infty}^{\infty} f(x)\left(\frac{\partial p(x,t)}{\partial t} - \frac{\sigma^2}{2}\frac{\partial^2 p(x,t)}{\partial x^2}\right)dx = 0.$$

Since the test function $f(x)$ is arbitrary, the term inside the parentheses must be identically zero everywhere. Therefore, we arrive at the Fokker-Planck equation for pure Brownian motion:

$$\frac{\partial p(x,t)}{\partial t} = \frac{\sigma^2}{2}\frac{\partial^2 p(x,t)}{\partial x^2}.$$

Notice that this is exactly the classical heat equation (or diffusion equation) with a diffusion coefficient of $D = \sigma^2/2$.

Verifying the Gaussian Solution

We now verify that the normal distribution $\mathcal{N}(0, \sigma^2 t)$ solves this Fokker-Planck equation. The probability density function is:

$$p(x,t) = \frac{1}{\sqrt{2\pi\sigma^2 t}}\exp\left(-\frac{x^2}{2\sigma^2 t}\right).$$

Computing the Time Derivative Using the product rule and chain rule, we differentiate $p(x,t)$ with respect to t:

$$\begin{aligned}\frac{\partial p}{\partial t} &= \left(-\frac{1}{2}(2\pi\sigma^2 t)^{-3/2}(2\pi\sigma^2)\right)\exp\left(-\frac{x^2}{2\sigma^2 t}\right) \\ &\quad + \frac{1}{\sqrt{2\pi\sigma^2 t}}\exp\left(-\frac{x^2}{2\sigma^2 t}\right)\left(-\frac{x^2}{2\sigma^2}\right)\left(-t^{-2}\right) \\ &= p(x,t)\left(-\frac{1}{2t}+\frac{x^2}{2\sigma^2 t^2}\right).\end{aligned}$$

Computing the Spatial Derivatives Next, we compute the first and second partial derivatives with respect to x:

$$\frac{\partial p}{\partial x} = p(x,t)\left(-\frac{2x}{2\sigma^2 t}\right) = -\frac{x}{\sigma^2 t}p(x,t).$$

Applying the product rule for the second derivative:

$$\begin{aligned}\frac{\partial^2 p}{\partial x^2} &= \frac{\partial}{\partial x}\left(-\frac{x}{\sigma^2 t}p(x,t)\right) \\ &= -\frac{1}{\sigma^2 t}p(x,t) - \frac{x}{\sigma^2 t}\frac{\partial p(x,t)}{\partial x} \\ &= -\frac{1}{\sigma^2 t}p(x,t) - \frac{x}{\sigma^2 t}\left(-\frac{x}{\sigma^2 t}p(x,t)\right) \\ &= p(x,t)\left(-\frac{1}{\sigma^2 t}+\frac{x^2}{\sigma^4 t^2}\right).\end{aligned}$$

Checking the PDE We substitute the second spatial derivative back into the right side of the Fokker-Planck equation:

$$\begin{aligned}\frac{\sigma^2}{2}\frac{\partial^2 p}{\partial x^2} &= \frac{\sigma^2}{2}\left[p(x,t)\left(-\frac{1}{\sigma^2 t}+\frac{x^2}{\sigma^4 t^2}\right)\right] \\ &= p(x,t)\left(-\frac{\sigma^2}{2\sigma^2 t}+\frac{\sigma^2 x^2}{2\sigma^4 t^2}\right) \\ &= p(x,t)\left(-\frac{1}{2t}+\frac{x^2}{2\sigma^2 t^2}\right).\end{aligned}$$

This exactly matches our computed expression for $\frac{\partial p}{\partial t}$. Therefore, the Gaussian density $\mathcal{N}(0,\sigma^2 t)$ satisfies the Fokker-Planck equation.

Problem 5

Implement the basic Brownian motion construction starting with fixed values of $\Delta X > 0$, $\Delta t > 0$.

a) Starting with $\Delta X = 1/2, \Delta t = 1/3$, write code that simulates $X(t)$ for $t = 0, \Delta t, 2\Delta t, ..., 10$. Illustrate this code with a graph of multiple sample runs of $X(t)$.

The initial construction uses a basic symmetric random walk with fixed step sizes $\Delta X = 1/2$ and $\Delta t = 1/3$. At each time increment, the process jumps by $\pm\Delta X$ with equal probability. We use a post-step plot to accurately represent the discrete nature of the simulation, showing that the position $X(t)$ remains constant between time steps. The simulation runs are shown in Figure 7.

```
import numpy as np
import matplotlib.pyplot as plt

def sim_rw(dx, dt, T, n_paths=12, seed=42):
        Simulates symmetric random walk paths.
    np.random.seed(seed)
    steps = int(round(T / dt))
    t = np.linspace(0, T, steps + 1)
    paths = [
        np.r_[0, np.cumsum(np.random.choice([-dx, dx], size=steps))
    ]
        for _ in range(n_paths)
    ]
    return t, paths

sigma, T = 2.0, 10.0
t_a, paths_a = sim_rw(dx=0.5, dt=1/3, T=T)

fig, ax = plt.subplots(figsize=(8, 4))
for x in paths_a:
    ax.step(t_a, x, where= post , alpha=0.7, linewidth=1.3)
ax.axhline(0, color= black , linestyle= -- , alpha=0.5)
ax.set(xlabel= t , ylabel= X(t) , title= Fixed \(\Delta X = 0.5, \
    Delta t = 1/3\) )
plt.show()
```

b) Tie the two increments together with the formula $\Delta x = \sigma\sqrt{\Delta t}$, $\sigma = 2$. Draw two more graphs of sample functions $X(t)$ for increasingly small values of Δt.

To converge toward a continuous Brownian motion, we must tie the spatial increment to the time increment. Setting $\Delta X = \sigma\sqrt{\Delta t}$ ensures that the variance accumulated over any time t scales linearly, $\mathrm{Var}[X(t)] = \sigma^2 t$, which prevents the process from blowing up or flattening out as $\Delta t \to 0$. As Δt decreases, the paths become visibly smoother and approximate the continuous Wiener process. This is shown in Figure 8.

```
# Simulate with tied increments for decreasing dt
t_b1, paths_b1 = sim_rw(sigma * np.sqrt(0.10), 0.10, T, seed=7)
t_b2, paths_b2 = sim_rw(sigma * np.sqrt(0.01), 0.01, T, seed=7)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4), sharey=True)

for x in paths_b1:
    ax1.plot(t_b1, x, alpha=0.65, linewidth=1.0)
```

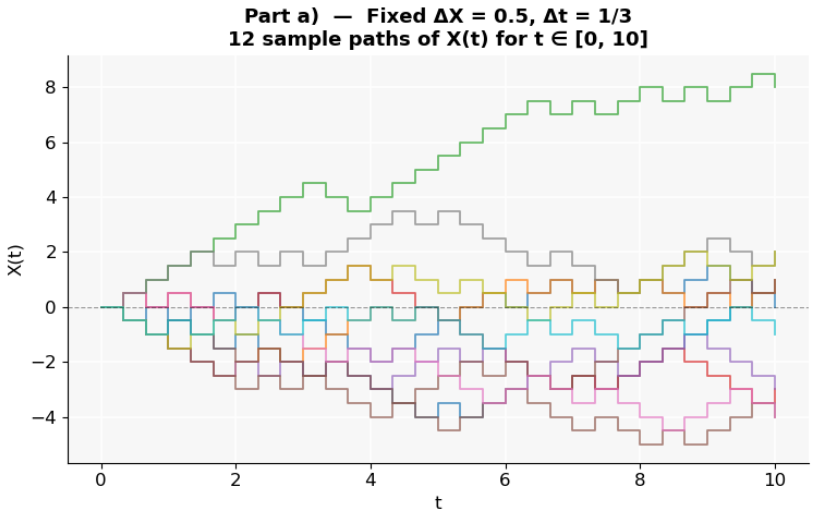

Figure 7: Simulations at fixed ΔX and Δt.

```
ax1.set(xlabel= t , ylabel= X(t) , title= \(\Delta t = 0.1\) )
ax1.axhline(0, color= black , linestyle= -- , alpha=0.5)

for x in paths_b2:
    ax2.plot(t_b2, x, alpha=0.65, linewidth=1.0)
ax2.set(xlabel= t , title= \(\Delta t = 0.01\) )
ax2.axhline(0, color= black , linestyle= -- , alpha=0.5)

plt.tight_layout()
plt.show()
```

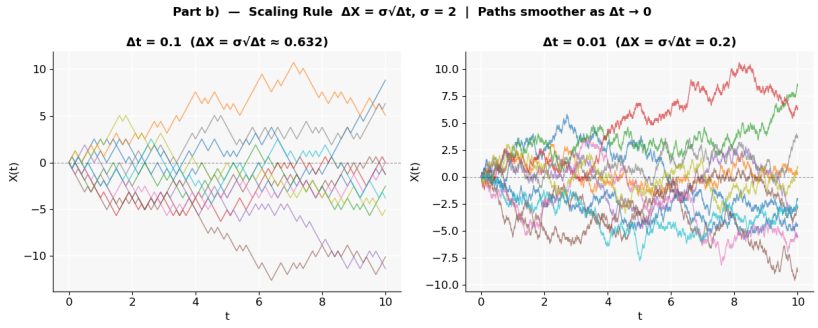

Figure 8: Simulations after applying the scaling rule.

c)Show a numerical distribution of $X(10)$, and compare it with the theoretical Gaussian distribution, for a small value of Δt.

By the Central Limit Theorem, the sum of independent increments causes the endpoint $X(T)$ to converge in distribution to a Gaussian, specifically $\mathcal{N}(0, \sigma^2 T)$. For $T = 10$ and $\sigma = 2$, the theoretical standard deviation is $2\sqrt{10} \approx 6.32$. Simulating a large number of independent paths at a very small $\Delta t = 0.001$ demonstrates an excellent match between the empirical histogram and the theoretical probability density function, as shown in Figure 9.

```
from scipy.stats import norm

dt_c = 0.001
dx_c = sigma * np.sqrt(dt_c)
steps_c = int(round(T / dt_c))
n_samp = 8000

# Vectorized simulation for X(10) endpoints
np.random.seed(99)
rng = np.random.default_rng(99)
X_T = np.zeros(n_samp)
batch_size = 200

# Batch to manage memory
for start in range(0, n_samp, batch_size):
    end = min(start + batch_size, n_samp)
    inc = rng.choice([-dx_c, dx_c], size=(end - start, steps_c))
    X_T[start:end] = inc.sum(axis=1)

th_std = sigma * np.sqrt(T)
x_range = np.linspace(-4 * th_std, 4 * th_std, 500)

fig, ax = plt.subplots(figsize=(8, 4))
ax.hist(X_T, bins=70, density=True, alpha=0.7, edgecolor= white ,
    label= Simulated X(10) )
ax.plot(x_range, norm.pdf(x_range, 0, th_std), color= red ,
    linewidth=2.5,
        label=f Theory \(\mathcal{{N}}(0, \sigma^2 T)\) )
ax.set(xlabel= X(10) , ylabel= Density , title= Distribution of X
    (10) )
ax.legend()
plt.show()
```

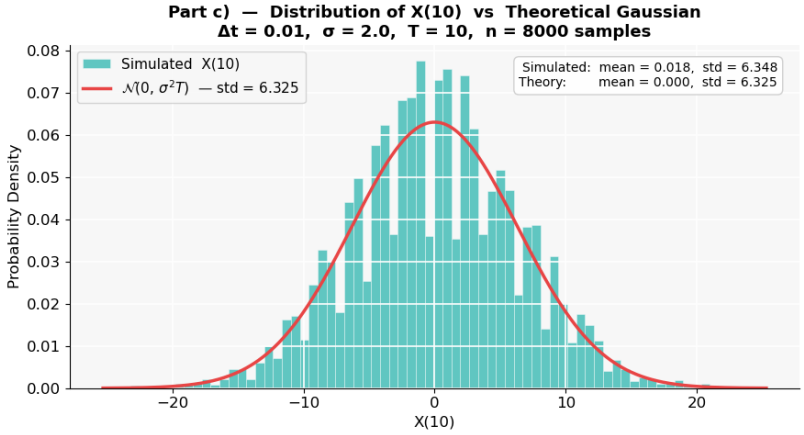

Figure 9: Convergence to normal distribution.